

Vue d'architecture de deployment (MSPR3 / TPRE601)

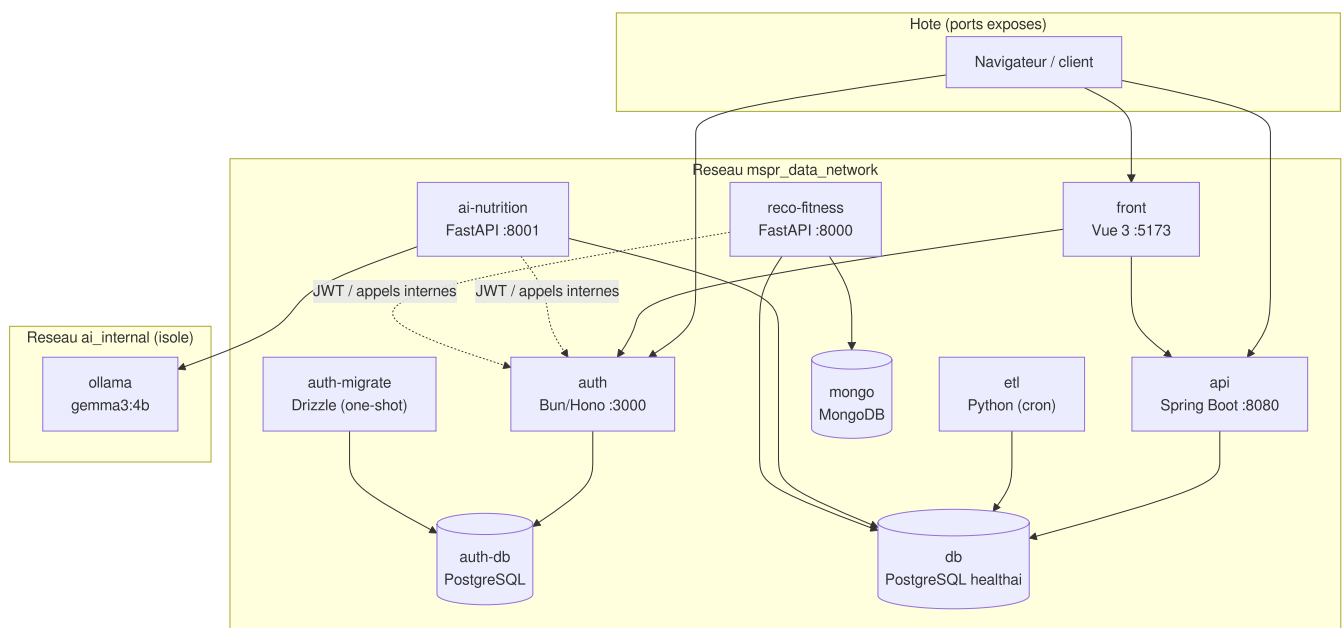
Ce document décrit l'architecture de deployment de la plateforme MSPR HealthAI Coach telle qu'elle est orchestrée par Docker Compose dans le dépôt `MSPR-Deploy`. Il se concentre sur les conteneurs, les réseaux, les ports, le flux d'authentification et la stack d'observabilité. Pour le pas-a-pas de démarrage (prérequis, `bootstrap.sh`, commandes utiles, dépannage), se reporter au `README.md`.

L'application mobile est prise en charge séparément par un autre membre de l'équipe. C'est un client de la plateforme (elle ne fait pas partie de la stack orchestrée) : elle est décrite dans `application-mobile.md`.

1. Conteneurs et réseaux

La stack compte 8 services applicatifs plus les bases de données et le conteneur Ollama. Deux réseaux bridge :

- `mspr_data_network` : réseau principal, tous les services y communiquent.
- `ai_internal` : réseau isolé réservant Ollama à `ai-nutrition` (Ollama n'expose aucun port sur l'hôte et n'est joignable que par `ai-nutrition`).



Notes de lecture :

- `ai-nutrition` est le seul service rattaché aux deux réseaux (`mspr_data_network` + `ai_internal`). Ollama n'est présent que sur `ai_internal`.
- `auth-migrate` est un conteneur one-shot (`restart: no`) : il applique les migrations Drizzle sur `auth-db` puis s'arrête. `auth` ne démarre qu'après sa completion.
- Les dépendances de démarrage sont gérées via `depends_on` + `healthcheck` : `api`, `etl`, `ai-nutrition` et `reco-fitness` attendent que `db` soit `healthy` ; `reco-fitness` attend aussi `mongo` (`healthy`) et `auth`.
- Le volume `etl_data` est partagé : l'`etl` y écrit (`/app/data`), l'`api` le lit en lecture seule (`/app/etl-data:ro`).

2. Ports exposes

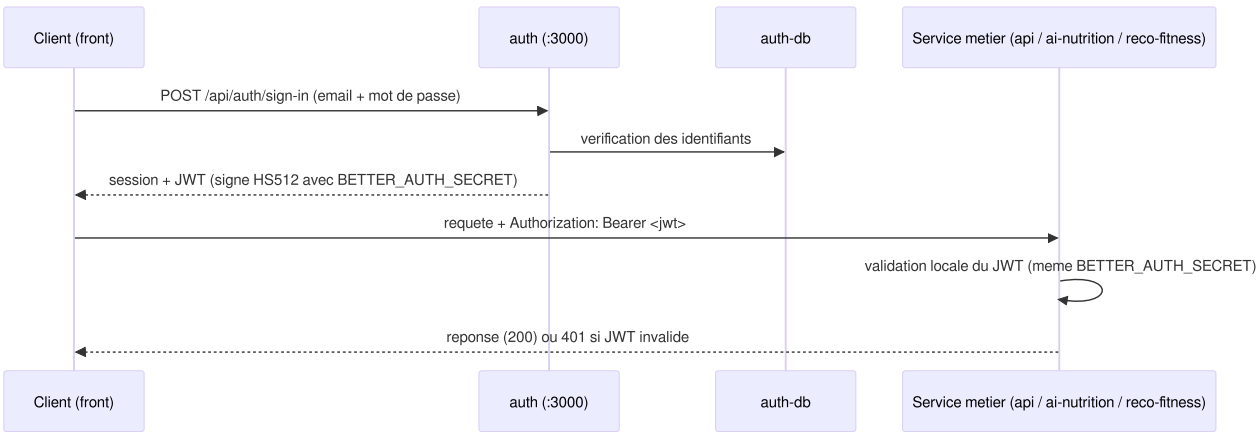
Les ports des bases de donnees sont bindes sur `127.0.0.1` (accessibles uniquement depuis l'hote, pas depuis le reseau). Les services applicatifs exposent leur port sur toutes les interfaces. Ollama n'expose aucun port.

Service	Mapping hote -> conteneur	Bind	Acces
front	<code>5173:5173</code>	toutes interfaces	Dashboard Vue 3
auth	<code>3000:3000</code>	toutes interfaces	Endpoints better-auth
api	<code>8080:8080</code>	toutes interfaces	API REST + Swagger
ai-nutrition	<code>8001:8001</code>	toutes interfaces	FastAPI (classification + plan repas)
reco-fitness	<code>8002:8000</code>	toutes interfaces	FastAPI (recommandations)
db (PostgreSQL metier)	<code>127.0.0.1:5434:5432</code>	localhost uniquement	base <code>healthai</code>
auth-db (PostgreSQL auth)	<code>127.0.0.1:5433:5432</code>	localhost uniquement	base <code>auth_db</code>
mongo (MongoDB)	<code>127.0.0.1:27018:27017</code>	localhost uniquement	base <code>reco_fitness</code>
ollama	aucun	reseau <code>ai_internal</code>	non joignable depuis l'hote
etl, auth-migrate	aucun	reseau interne	pas de port

Ports ajoutes par l'overlay de monitoring : voir section 4.

3. Flux d'authentification (JWT)

L'authentification est entierement deleguee au service `auth` (better-auth). Les services metier ne gerent pas de comptes : ils valident le JWT a l'aide du secret HMAC partage `BETTER_AUTH_SECRET` (algorithme HS512), injecte par variable d'environnement dans `auth`, `api`, `ai-nutrition` et `reco-fitness`.

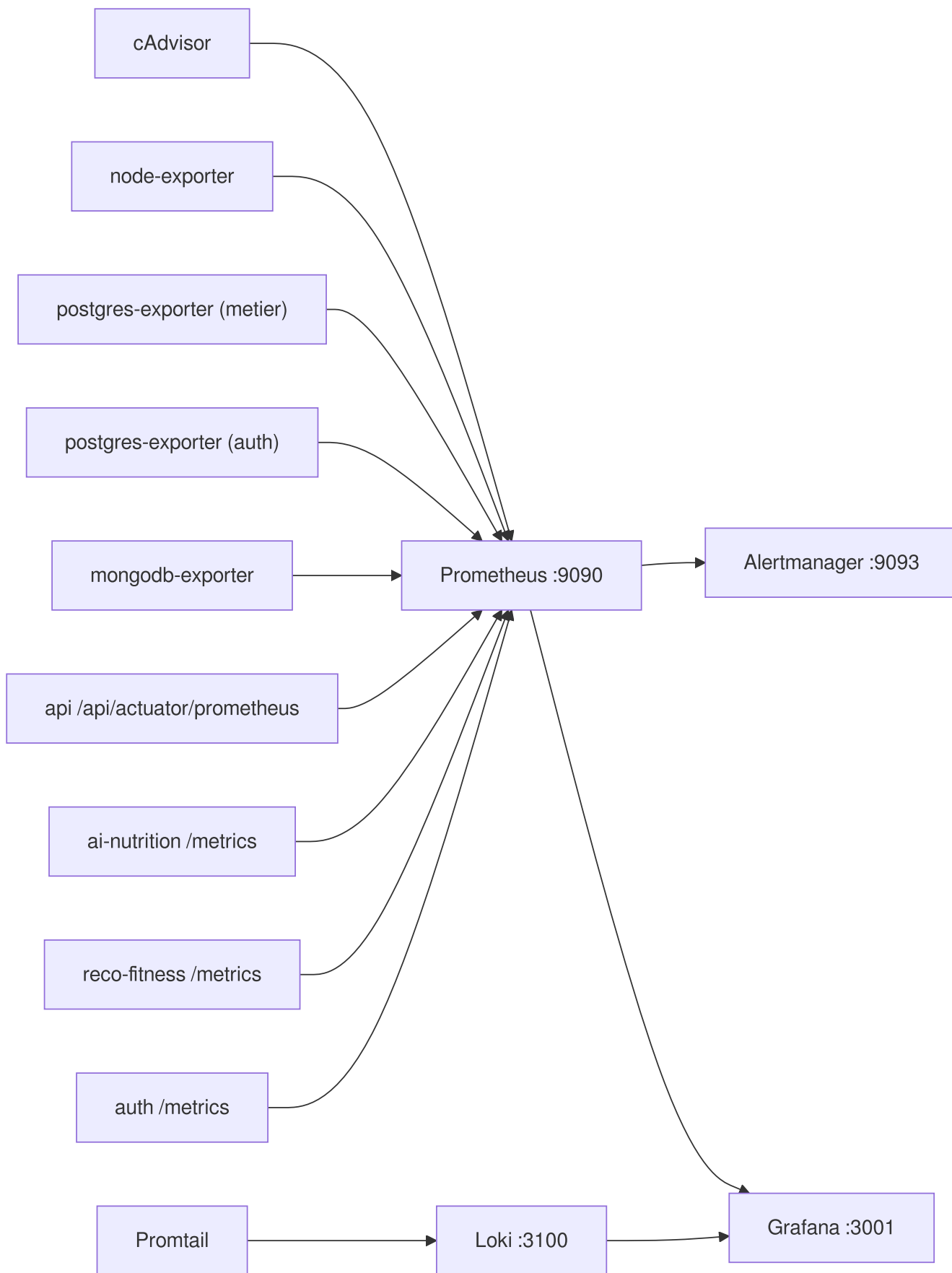


Points cles :

- La validation du JWT est locale a chaque service (verification de signature), sans aller-retour systematique vers `auth`.
- `ai-nutrition` et `reco-fitness` connaissent aussi l'URL interne de `auth` (`AUTH_API_URL=http://mspr-auth-service:3000`) pour les appels necessitant le service.
- Le secret est genere par `bootstrap.sh` (il n'y a plus de valeur par default versionnee).

4. Observabilite (qui scrape quoi)

La supervision est fournie par l'overlay `docker-compose.monitoring.yml`, qui s'ajoute a la stack sans modifier les services applicatifs. Detail complet dans `monitoring/README.md`.



Cibles scrapees par Prometheus (`monitoring/prometheus/prometheus.yml`) :

Job Prometheus	Cible	Donnees
cadvisor	cadvisor:8080	metriques par conteneur (CPU, RAM, reseau, IO)
node-exporter	node-exporter:9100	metriques de l'hote
postgres-metier	postgres-exporter:9187	PostgreSQL base healthai
postgres-auth	postgres-exporter-auth:9187	PostgreSQL base auth_db
mongodb	mongodb-exporter:9216	MongoDB base reco_fitness
api	mspr-api:8080 /api/actuator/prometheus	HTTP (latence, debit, codes), JVM
ai-nutrition	mspr-ai-nutrition:8001 /metrics	requetes HTTP (RED)
reco-fitness	mspr-reco-fitness:8000 /metrics	requetes HTTP (RED)
auth	mspr-auth-service:3000 /metrics	requetes HTTP (RED)

Les logs de tous les conteneurs sont collectes par Promtail (via le socket Docker) et poussees vers Loki, interrogeables dans Grafana. Trois regles d'alerte (`monitoring/prometheus/alerts.yml`) sont evaluees par Prometheus et routees vers Alertmanager :

Alerte	Condition	Severite
CibleInjoignable	up == 0 pendant 1 min	critical
ConteneurCpuEleve	> 0.9 coeur CPU pendant 5 min	warning
ConteneurMemoireElevee	> 2 Go de RAM pendant 5 min	warning

Ports ajoutes par l'overlay : Prometheus 9090 , Alertmanager 9093 , Grafana 3001 , Loki 3100 , cAdvisor 8085 , node-exporter 9100 , postgres-exporter 9187 , postgres-exporter-auth 9188 , mongodb-exporter 9216 .

Les metriques applicatives (api, ai-nutrition, reco-fitness, auth) ne sont visibles qu'apres reconstruction des images (l'instrumentation vit dans le code source). Les metriques d'infrastructure (cAdvisor, node-exporter, exporters de bases) sont actives des le lancement.

5. Configurations de deployment et point d'entree

La stack se declina en plusieurs configurations et overlays, tous detailles dans `CONFIGS.md` . Ils se combinent avec un compose de base (`docker-compose.yml` en mode prod / images GHCR, ou `docker-compose.dev.yml` en mode dev / build local).

Configuration	Overlay	Objectif
Complete	<code>docker-</code> <code>compose.monitoring.yml</code>	tous les services + IA generative + monitoring complet
Offline	<code>docker-</code> <code>compose.offline.yml</code>	demo sans internet : Ollama local, Auth sans email (<code>AUTH_OFFLINE</code>)
Performance	<code>docker-</code> <code>compose.performance.yml</code>	matériel modeste : limites CPU/RAM, monitoring simplifié, IA lourde optionnelle
Exposition publique	<code>docker-</code> <code>compose.traefik.yml</code>	reverse proxy Traefik + TLS, routage <code>*.zespri.duckdns.org</code> (front, api, auth, ai, reco, grafana)
Classification photo	<code>docker-compose.vision.yml</code>	<code>ai-nutrition</code> sur le backend Mistral vision (route A1)
Deploiement continu	<code>docker-</code> <code>compose.watchtower.yml</code>	Watchtower : redéploiement automatique des services backend sur nouvelle image GHCR (CD)

Les trois premières lignes sont des configurations alternatives ; les trois suivantes sont des overlays composables. En production sur le serveur, ils sont combinés (`monitoring` + `traefik` + `vision` + `watchtower`) : la plateforme est exposée en HTTPS derrière Traefik (réseau externe `proxy_net`), supervisée, et maintenue à jour automatiquement par Watchtower (boucle CI/CD complète, voir `CICD.md`).

Point d'entrée unique : `./bootstrap.sh` (mode prod par défaut, `--dev` pour cloner les sources et builder localement). Le script crée `.env` , génère `BETTER_AUTH_SECRET` , déchiffre les secrets puis lance la stack. Le pas-à-pas complet est dans le `README.md` .

Resilience : les scripts `scripts/backup.sh` , `scripts/restore.sh` et `scripts/clean.sh` gèrent la sauvegarde (`pg_dump` des deux bases PostgreSQL + `mongodump`), la restauration et la remise à zéro.